PYTHON WITH AI • PROJECT WALKTHROUGH

Student Performance Analytics System

A beginner-friendly guide to understanding, building, documenting, and submitting the project

Based on: EWB Courses — Python Project Assignment for Python with AI Students

Required technologies: Python, NumPy, Pandas

Deadline stated in assignment: 15 October 2026 (on or before)

How to use this guide: Read the assignment requirements first, then build the project step by step. Example rules and code snippets in this guide are learning examples; adapt and test them yourself.

1. What the assignment asks you to build

The assigned project is a Python-based **Student Performance Analytics System**. It reads a dataset of student records, processes the marks and attendance, and presents useful summaries about student and class performance.

In simple Hinglish: students ka data file se read karna, marks par calculations karna, grades/pass-fail nikalna, subjects ka average dekhna, aur top-performing students identify karna.

Project details from the assignment

Item	Assignment requirement
Project title	Student Performance Analytics System
Technology	Python, NumPy, Pandas
Concepts covered	Variables, data types, loops, functions, NumPy, Pandas, data analysis
Dataset size	At least 15–20 student records
Submission	Project + GitHub repository + documentation + Google Form link submission
Deadline	15 October 2026 (on or before)

Main goal

- Calculate total marks and average marks for every student.
- Assign grades using a clearly defined grading rule.
- Identify pass/fail students using a clearly defined condition.
- Find highest, lowest, and average performance.
- Analyze performance subject by subject.
- Identify top-performing students.
- Present results so a reader can understand them without inspecting the code.

The assignment explicitly says advanced machine learning is **not required**. Focus on the Python, NumPy, Pandas, and analysis concepts already taught in class.

2. Understand the data you will work with

The assignment suggests a dataset containing fields such as Student ID, Name, Department, subject marks, and Attendance. You can store these records in a CSV file named **students.csv**.

Column	Meaning	Example only
Student_ID	Unique identifier for each student	101
Name	Student name	Aarav
Department	Student's department	CSE
Maths	Marks in Maths	82
Science	Marks in Science	76
English	Marks in English	88
Computer	Marks in Computer	91
Attendance	Attendance percentage or count (define which)	92%

Important: The exact subject names and exact grading scale are not prescribed in the brief. Choose sensible columns and document your choices. The examples in this guide are illustrative, not official rules.

What is a CSV file?

CSV means **Comma-Separated Values**. It is a plain-text table format. Each line usually represents one record, and commas separate the values.

```
Student_ID,Name,Department,Maths,Science,English,Computer,Attendance
101,Aarav,CSE,82,76,88,91,92
102,Meera,IT,69,84,79,86,88
```

Hinglish: CSV ko Excel jaisi table samajh sakte ho. Pandas is file ko read karke Python ke andar DataFrame banata hai.

What is a Pandas DataFrame?

A **DataFrame** is a table-like data structure with rows and columns. Each row can represent one student, and each column stores a field such as name, department, marks, or attendance.

```
import pandas as pd

students = pd.read_csv("students.csv")
print(students.head())       # first 5 rows
print(students.shape)       # (rows, columns)
print(students.columns)     # column names
print(students.info())      # data types and missing values
```

Make sure **students.csv** is saved in the correct folder, or provide the correct file path. Check that numeric marks are stored as numbers, not accidental text.

3. The Python concepts you need

Variables and data types

A variable is a name used to store a value. Common types in this project are **int** for whole-number marks or IDs, **float** for averages, **str** for names/departments, and **bool** for True/False conditions.

```
student_name = "Aarav"   # str
maths_marks = 82          # int
average_marks = 84.25     # float
is_pass = True            # bool
```

Conditions: if / elif / else

Conditions let your program make decisions. For example, if the average meets the pass threshold, mark the student as Pass; otherwise mark them as Fail. The assignment asks you to define the condition clearly.

```
if average_marks >= 40:
    result = "Pass"
else:
    result = "Fail"
```

The 40-mark threshold above is only an example. Your project must state the rule you actually choose, including whether passing depends on the average, every subject, or both.

Loops

A loop repeats an action for multiple items. You might use a loop to process each student or print each student's result. Pandas can also perform many operations on whole columns without a manual loop.

```
for name in students["Name"]:
    print(name)
```

Functions

A function is a reusable block of code that performs a task. Functions keep calculations organized and avoid repeating the same logic. The assignment specifically expects reusable functions for calculations and processing.

```
def calculate_average(marks):
    return sum(marks) / len(marks)

print(calculate_average([82, 76, 88, 91]))
```

NumPy

NumPy supports efficient numerical calculations. Use it where appropriate for totals, averages, minimums, maximums, or other numerical operations. You do not need to force NumPy into every line of code.

```
import numpy as np

marks = np.array([82, 76, 88, 91])
print(np.sum(marks))
print(np.mean(marks))
print(np.max(marks))
print(np.min(marks))
```

Pandas

Pandas helps read, organize, filter, summarize, and analyze tabular data. It is especially useful for the student dataset.

```
subject_columns = ["Maths", "Science", "English", "Computer"]
students["Total"] = students[subject_columns].sum(axis=1)
students["Average"] = students[subject_columns].mean(axis=1)
print(students[["Name", "Total", "Average"]])
```

Understanding axis=1: in this example, Pandas calculates across the selected subject columns for each row/student. Check that the subject columns contain numeric marks and that the maximum marks per subject are consistent.

4. Required calculations and analysis

A. Total marks per student

Add the student's marks across all selected subjects. If there are four subjects, the total is the sum of those four marks. State the subjects included in the total.

B. Average marks per student

Average = total marks ÷ number of subjects. For example only, marks 80, 70, 90, 60 give total 300 and average 75.

C. Grades

Create a grading rule and use it consistently. The assignment requires a clearly defined rule but does not provide an official grading scale.

Example average	Example grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

This table is a suggested example only. You can choose a different grading scale if appropriate, but include the exact scale in your documentation.

D. Pass/fail status

Define a clear condition and explain it. One possible rule is passing if the average is at least 40; another is requiring a minimum score in every subject. Do not leave the logic ambiguous.

E. Highest, lowest, and overall class performance

```
print("Highest average:", students["Average"].max())
print("Lowest average:", students["Average"].min())
print("Class average:", students["Average"].mean())
```

You should also identify which student has the highest average, not just the numerical highest value.

```
top_student = students.loc[students["Average"].idxmax()]
print(top_student[["Name", "Average"]])
```

F. Subject-wise performance

Compare the average marks of each subject to see which subject has the strongest or weakest class performance.

```
subject_averages = students[subject_columns].mean()
print(subject_averages)
print("Best average subject:", subject_averages.idxmax())
print("Lowest average subject:", subject_averages.idxmin())
```

G. Count pass/fail students and identify top performers

```
# Example only: assumes a Pass column has already been created
print(students["Result"].value_counts())

# Sort from highest to lowest average
print(students.sort_values("Average", ascending=False).head(5))
```

Use the same grade and pass/fail rules everywhere. Test boundary values (for example, exactly at the pass mark) so the result is predictable.

5. A possible project structure

The assignment gives these suggested names; exact filenames may vary as long as the repository is clear and easy to understand.

File/folder	Purpose
main.py	Runs the project and displays the analysis.
students.csv	Stores 15–20 or more student records.
functions.py (optional)	Contains reusable calculation or processing functions.
README.md	Explains the project and how to run it.
Project_Documentation.pdf or .docx	Explains objective, dataset, implementation, outputs, and outcome.

```
student-performance-analytics/
    main.py
    students.csv
    functions.py              # optional
    README.md
    Project_Documentation.pdf # example name
```

Keep files organized, use meaningful names, add comments where helpful, and remove unnecessary files before uploading to GitHub.

6. Step-by-step implementation workflow

Step 1 — Prepare the dataset

Create students.csv with at least 15–20 realistic student records. Include student details, marks, and attendance. Avoid using only one or two rows.

Step 2 — Read and inspect the data

Use Pandas read_csv. Display the first rows, column names, number of records, and basic information. Look for missing values or incorrect data types.

Step 3 — Process the data

Use functions, loops, conditions, NumPy, and Pandas where appropriate. Calculate totals, averages, grades, and pass/fail status.

Step 4 — Perform analysis

Answer practical questions: Which student has the highest average? Which subject has the best average? How many students passed? What is the class average?

Step 5 — Present final output

Print results with clear labels and a readable layout. Output should make sense without requiring the reader to inspect the source code.

Step 6 — Test and correct errors

Run the main program more than once. Check calculations manually for a few records and test edge cases such as the minimum passing score.

Step 7 — Prepare GitHub repository

Create a repository with a clear name such as student-performance-analytics, then upload the Python files, dataset, README, and documentation.

Step 8 — Submit the link

Open the repository in your browser, verify all required files are visible, copy its URL, and paste it into the Google Form provided by EWB Courses.

Step 9 — Finish documentation

Write the required documentation, add relevant screenshots if useful, push the documentation to GitHub, and verify the final repository.

7. GitHub: what to do and why

GitHub hosts your project repository online so another person can inspect the files. The assignment requires a repository and asks you to submit its URL through the Google Form.

- Sign in to GitHub and create a new repository with a clear name.
- Set repository visibility/accessibility as instructed by your trainer.
- Upload main.py, the dataset, README.md, and the required documentation; include functions.py if you use it.
- Commit and push the complete project.
- Open the repository in a browser and confirm the files are visible and the project can be followed.
- Copy the repository URL and submit it in the EWB Google Form.
- After adding documentation, push the updated files and re-check the repository.

README.md should help another person: understand the project purpose, know which files are included, see the required libraries, and know how to run the main program. These are useful README suggestions; the assignment specifically requires a README file.

8. Project documentation: required sections

The assignment asks you to prepare documentation explaining what you built, how it was implemented, and what result was obtained. Use these sections:

Section	What to include
1. Project Title	Project name and student details.
2. Objective	Why the project was created and what problem it addresses.
3. Technologies Used	Python, NumPy, Pandas, and other tools actually used.
4. Dataset Description	Columns, important fields, and approximate number of records.

Section	What to include
5. Implementation	How you load data, process it, use functions/loops, and perform analysis.
6. Key Features	List the features that you have actually implemented.
7. Output / Screenshots	Add screenshots of important outputs if useful; screenshots are optional in the brief.
8. Final Outcome	Explain what the completed project successfully achieved, based on real results.
9. Challenges & Learning	Describe problems you faced, how you solved them, and what you learned.
10. GitHub Repository	Add the final repository URL.

Do not write imaginary results. Run your own code first, then use its actual output in the documentation.

9. Assessment: what the evaluator may look for

Assessment area	Focus in the assignment
Python fundamentals	Correct use of functions, loops, conditions, and data handling.
NumPy	Appropriate use for numerical operations.
Pandas	DataFrame creation/loading, filtering, analysis, and useful operations.
Project logic	Correct calculations, conditions, and meaningful results.
Code quality	Readable structure, clear names, organization, and basic comments.
GitHub	Complete repository, correct files, and working README.
Documentation	Clear objective, implementation, outputs, and final outcome.

10. Final submission checklist

■ Project completed and tested

■ Student dataset included

■ NumPy and Pandas used appropriately

■ Functions and loops used where required

■ GitHub repository created

■ Complete project pushed to GitHub

■ README.md added

■ GitHub repository link submitted through the Google Form

■ Documentation completed with objectives, implementation, and final outcome

■ Documentation/screenshots added to the GitHub repository

■ Final repository checked before the deadline

Deadline: 15 October 2026, on or before, as stated in the assignment.

11. Beginner-friendly viva / self-check questions

Q. What is the objective of your project?

Answer: To analyze student marks and attendance data and produce meaningful performance summaries.

Q. Why did you use Pandas?

Answer: To load, organize, filter, and analyze student data in a tabular DataFrame.

Q. Why did you use NumPy?

Answer: For numerical operations such as sum, mean, maximum, and minimum where appropriate.

Q. What is a DataFrame?

Answer: A two-dimensional table-like structure with rows and columns.

Q. Why use functions?

Answer: To reuse calculations, reduce repeated code, and keep the program organized.

Q. Where are loops useful?

Answer: When repeating a task for multiple students or records.

Q. How do you calculate an average?

Answer: Add the marks and divide by the number of subjects.

Q. How do you assign a grade?

Answer: Apply a clearly documented grading rule to the student's marks or average.

Q. How do you identify the top performer?

Answer: Compare the average marks and select the student with the highest average.

Q. What does subject-wise analysis tell you?

Answer: It compares average performance across subjects and shows stronger or weaker subjects.

Q. Why is testing important?

Answer: It helps find incorrect calculations, data issues, and errors before submission.

Q. What is GitHub used for in this assignment?

Answer: To host the project files and provide a repository link for submission.

12. Common mistakes to avoid

- Using fewer than the required 15–20 student records.
- Calculating an average using the wrong number of subjects.
- Including Student_ID, Name, Department, or Attendance in the marks total by mistake.
- Using marks stored as text instead of numeric values.
- Writing a grade/pass-fail rule without explaining it.
- Showing only code and not clear results.
- Forgetting README.md or project documentation.
- Submitting a repository URL before checking that the files are visible.
- Claiming features or results in the documentation that the program does not actually produce.

13. Final advice

Build the project in small steps: first make the dataset, then read it, then calculate totals and averages, then add grades and pass/fail logic, and finally add analysis and presentation. Test each step before moving to the next.

Hinglish summary: pehle data sahi banao, phir Pandas se load karo, NumPy/Pandas se calculations karo, functions aur conditions se logic organize karo, results clearly print karo, aur end mein GitHub + README + documentation + Google Form submission complete karo.

This guide explains the uploaded assignment. The assignment does not specify an exact dataset, official grading scale, exact subject list, or required output numbers; decide and document those choices, and confirm any trainer-specific instructions.